

Predicting Human Preferences in LLM Responses

A Comparative Study of DeBERTa, RoBERTa, and ELECTRA on the Chatbot Arena Task

May 16, 2026

Abstract

This report investigates the task of predicting which of two large language model (LLM) responses a human evaluator would prefer, framed as a three-way classification problem (model A wins, model B wins, or a tie). The study is based on the Kaggle “LLM Classification Finetuning” competition, whose data is derived from the Chatbot Arena platform. We describe the dataset, the preprocessing pipeline, and a shared Siamese-style architecture in which three different pretrained encoders—DeBERTa-v3 (extra small), RoBERTa-base, and ELECTRA-large—are fine-tuned and compared. We present validation scores and, more importantly, examine the per-class probability outputs each model assigns to the same test examples in order to understand what each model is actually doing.

1 Introduction

The Chatbot Arena platform [1] collects head-to-head preference judgements between large language models: a user submits a prompt, two anonymous models respond, and the user votes for the preferred response or declares a tie. The Kaggle competition *LLM Classification Finetuning* releases a subset of this preference data and asks for a model that outputs, for each test example, three probabilities summing to one: the probability that model A wins, the probability that model B wins, and the probability of a tie. Submissions are scored on multi-class log loss, which rewards both correct predictions and well-calibrated probabilities.

The problem has structure that distinguishes it from standard text classification: the input is a triple (prompt, response A, response B), and the answer depends on a *comparison* between the two responses. How that structure is encoded by the model is at least as important as the choice of underlying pretrained encoder. This report fixes a single architectural template—a Siamese network with a shared encoder—and varies only the backbone, in order to isolate the effect of that choice.

2 Data

2.1 Source and structure

The training set contains 57,477 pairwise preference judgements. Each row provides:

- a **prompt** given to both models,
- two responses **response_a** and **response_b** generated by two anonymous LLMs,
- the identity of the two models (for context only),
- three binary indicators **winner_model_a**, **winner_model_b**, and **winner_tie** encoding the ground-truth outcome.

The participating models in the released split include frontier systems such as GPT-4, GPT-3.5, Llama-2, Koala, and Mistral, so the data covers a wide range of response styles, lengths, and quality levels.

2.2 Preprocessing

Preprocessing was deliberately light, to preserve content distinctions that human evaluators are likely to weigh. The steps were:

1. whitespace normalisation;
2. removal of non-ASCII characters, on the assumption of English-only encoders;
3. construction, for each training row, of two paired strings: (prompt, response_a) and (prompt, response_b) — the same prompt thus appears in both inputs;
4. collapsing the three binary outcome columns into a single label in $\{0, 1, 2\}$ ($0 = A$ wins, $1 = B$ wins, $2 = \text{tie}$);
5. a stratified 90/10 split into training and validation sets, preserving the class distribution.

Tokenisation followed each backbone’s own conventions: byte-pair encoding for DeBERTa-v3 and RoBERTa, WordPiece for ELECTRA. For ELECTRA the prompt and response were additionally encoded with explicit segment IDs (segment 0 for the prompt, segment 1 for the response), matching the form on which the model was pretrained.

3 The Siamese Architecture

3.1 Motivation

A Siamese network [3] is a model in which two (or more) inputs are passed through the *same* encoder with *shared weights*, producing two embeddings that can then be compared by a small head. The pattern was originally introduced for signature verification, where the question is “do these two signatures come from the same person?”, and has since become a standard tool for any task that asks a question about two related inputs: sentence similarity, image retrieval, face verification, duplicate detection.

The key property is weight sharing. If two responses are encoded by *different* models, the two resulting embeddings live in different spaces and cannot be compared directly. Sharing weights forces both inputs through the same representational lens, so element-wise differences and similarities between the embeddings become meaningful.

For the preference task here, both inputs — (prompt, response A) and (prompt, response B) — are objects of the same type. A Siamese encoder is therefore a natural choice, and is also computationally efficient to train, because there is only one set of encoder weights to update.

3.2 Architecture overview

Figure 1 shows the flow of information through the model. The same encoder is applied twice, once to each (prompt, response) pair. The two resulting sequences of contextual embeddings are each collapsed to a single vector by masked mean pooling. The two pooled vectors are then fused into a richer representation that includes their difference and their element-wise product, which give the classifier explicit comparison features. A small multilayer perceptron with dropout produces the final three-way softmax distribution.

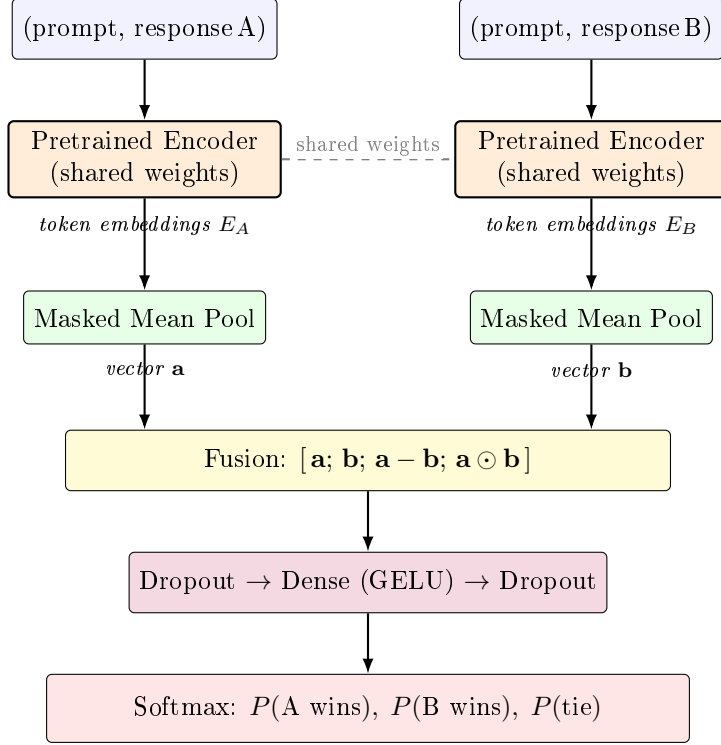


Figure 1: The Siamese architecture used in this study. The same pretrained encoder, with the same weights, is applied to each of the two (prompt, response) pairs. The two resulting embedding sequences are pooled to vectors $\mathbf{a}$ and $\mathbf{b}$, which are then fused with their difference and element-wise product to give the classification head explicit comparison features. The only thing that changes between experiments is the identity of the pretrained encoder.

3.3 Fusion details

The fused representation passed to the head is

$$\mathbf{f} = [\mathbf{a}; \mathbf{b}; \mathbf{a} - \mathbf{b}; \mathbf{a} \odot \mathbf{b}],$$

where $;$ denotes concatenation and $\odot$ denotes element-wise multiplication. Including $\mathbf{a} - \mathbf{b}$ gives the head direct access to the asymmetric “which is better” signal: if the two responses are encoded similarly, the difference is close to zero, which is a useful cue for a tie. Including $\mathbf{a} \odot \mathbf{b}$ supplies a symmetric agreement signal. This four-part fusion is a well-known recipe from the sentence-pair representation literature [7] and empirically outperforms simple concatenation.

3.4 Training procedure

All three models were trained with the same overall procedure: AdamW with weight decay 0.01, categorical cross-entropy with label smoothing ($\epsilon = 0.1$), an epoch budget of up to ten with early stopping on validation loss, and learning-rate reduction on plateau. Label smoothing both regularises the model and produces better-calibrated probabilities, which matters because the competition is scored on log loss rather than on accuracy.

4 Backbones Compared

All three backbones are stacks of standard transformer encoder blocks [2] and share the same high-level shape: input tokens are embedded, passed through N identical blocks, and emerge as a sequence of contextual representations. Figure 2 shows this shared skeleton.

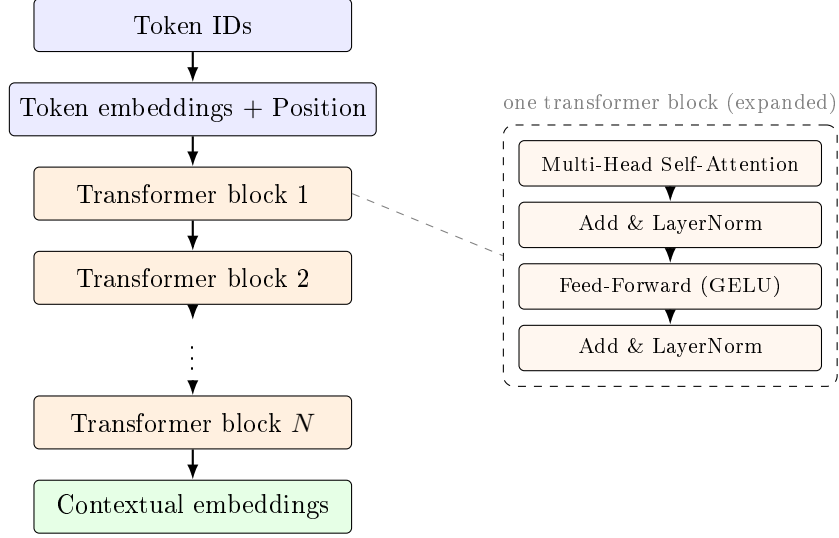


Figure 2: Shared transformer-encoder skeleton used by all three backbones. Input tokens are embedded, summed with positional information, and passed through N identical transformer blocks. Each block (right) consists of multi-head self-attention followed by a feed-forward layer, both wrapped in residual connections and layer normalisation. The three backbones differ in N and hidden size, and, more interestingly, in how attention and pretraining are realised — the subject of the next three figures.

RoBERTa-base

[5] RoBERTa is a thoroughly studied refinement of BERT that removes the next-sentence-prediction objective, trains on more data for longer, and uses dynamic masking. The *base* configuration has approximately 125M parameters, a hidden size of 768, and 12 layers, and uses byte-pair tokenisation.

Architecturally, RoBERTa uses the original BERT formulation. Content and position information are combined *additively* at the embedding layer: each token’s input vector is the sum of its content embedding and its absolute position embedding. The transformer blocks that follow operate on this combined representation, so attention scores are computed from a single mixed source. Figure 3 illustrates this.

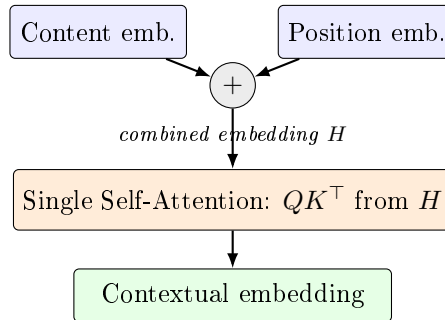


Figure 3: RoBERTa (and standard BERT) attention. Content and position embeddings are added together at the input. Attention scores are computed from queries and keys derived from this single combined vector — a single content-and-position attention term.

DeBERTa-v3 extra-small

[4] DeBERTa introduces *disentangled attention*, in which content and positional information are represented separately and combined into three distinct attention components, improving the model’s ability to reason about token positions. Version 3 is pretrained with the ELECTRA-style replaced-token-detection objective, which is more sample-efficient than masked language modelling. The *extra-small* configuration has approximately 22M parameters with a hidden size of 384 across 12 layers — the smallest of the three encoders compared here.

Figure 4 contrasts DeBERTa’s attention mechanism with RoBERTa’s. Each token is represented by two separate vectors — a content vector C and a relative-position vector P — and the attention score between tokens i and j is the sum of three terms: content-to-content (C_i, C_j), content-to-position ($C_i, P_{i \rightarrow j}$), and position-to-content ($P_{j \rightarrow i}, C_j$). The position-to-position term is omitted because it adds no information beyond the relative offset already captured.

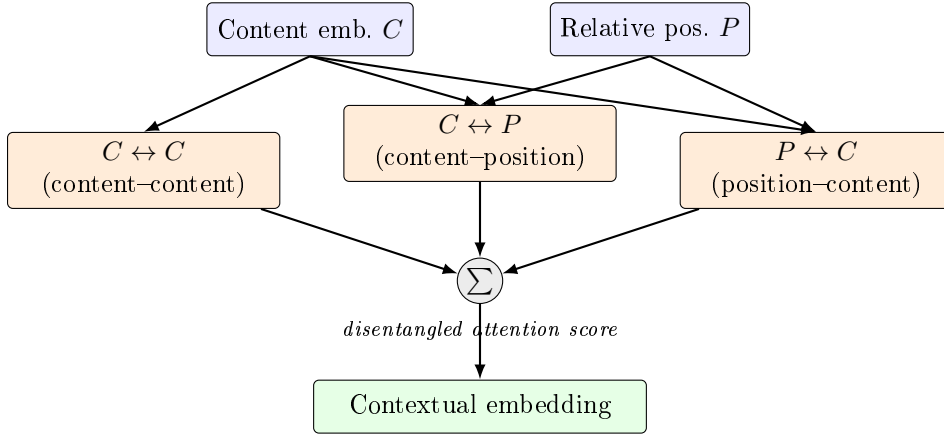


Figure 4: DeBERTa disentangled attention. Content and (relative) position embeddings are kept separate. The attention score between two tokens is the sum of three terms rather than the single term used by BERT/RoBERTa, which gives the model a cleaner signal about token positions and is the main source of DeBERTa’s accuracy gains.

ELECTRA-large

[6] ELECTRA introduced the replaced-token-detection pretraining objective that DeBERTa-v3 later adopted. The *large* discriminator has approximately 335M parameters with a hidden size of 1024 across 24 layers — the largest of the three encoders compared here. It uses WordPiece tokenisation and BERT-style segment encodings.

The internal structure of an ELECTRA block is essentially the same as RoBERTa’s. What makes ELECTRA distinctive is its *pretraining scheme*, sketched in Figure 5. A small *generator* model is trained to predict masked tokens (like ordinary masked-language-modelling). Its predictions, sampled and substituted back into the input, produce a plausibly-corrupted sequence. A separate, larger *discriminator* model is then trained to label each token as **original** or **replaced**. After pretraining, the generator is discarded and only the discriminator is used for downstream fine-tuning — which is what we do here.

Architectural specifications side by side

Table 1 sets out the configuration of the three encoders. The counts are taken from each model’s official release: the `microsoft/deberta-v3-xsmall` card on Hugging Face, the `roberta-base` configuration, and the `electra-large` specification published in the original Google Research repository.

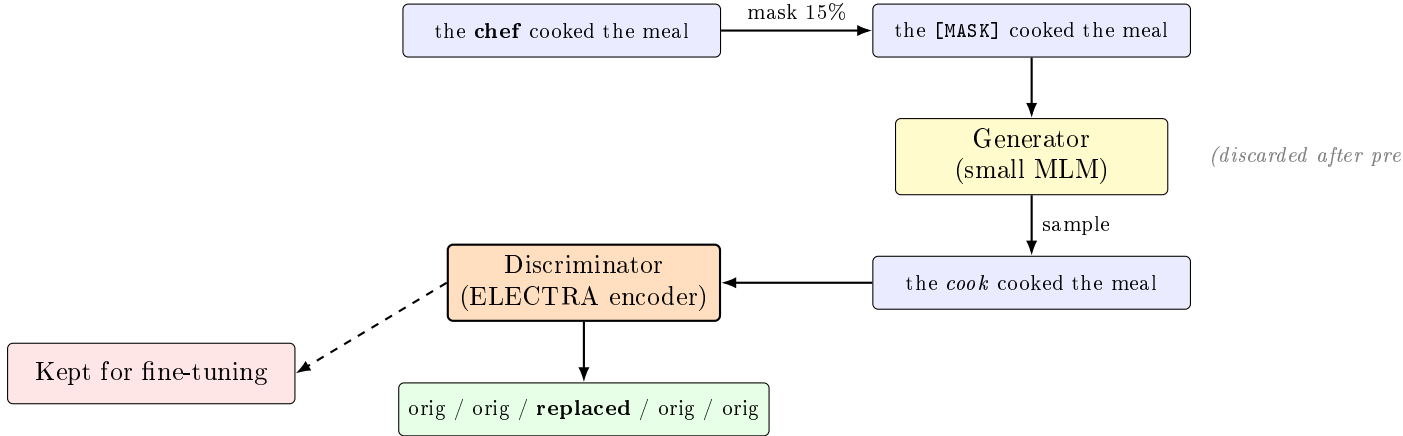


Figure 5: ELECTRA’s replaced-token-detection pretraining. A small generator proposes replacements for masked tokens; the larger discriminator is trained to identify which tokens have been replaced. The objective applies to *every* token rather than just the masked ones, which is the source of ELECTRA’s well-known sample efficiency. Only the discriminator survives into fine-tuning, where it is used as a standard transformer encoder.

A few observations follow directly from this table.

The per-head dimension is the same in all three. All three encoders use $H/A = 64$ dimensions per attention head, which is the standard choice in the BERT family. Width and head count therefore scale together: doubling the hidden size also doubles the number of heads.

Depth matters as much as width. ELECTRA-large doubles *both* the layer count and the hidden size relative to RoBERTa-base. Roughly speaking, parameter count in a transformer scales as $\mathcal{O}(N \cdot H^2)$, so doubling each dimension yields an eightfold increase in backbone parameters — which is consistent with the 86M $\rightarrow$ 335M jump in the table.

DeBERTa-v3-xs is small in backbone but large in embedding. The 128K SentencePiece vocabulary is shared with the full DeBERTa-v3 family and contributes 48M parameters to a model whose transformer stack accounts for only 22M. This is a deliberate design choice: a large vocabulary captures more of each word as a single token, reducing sequence length and improving downstream accuracy for a fixed compute budget. It also means that the “total parameter count” of ~ 70 M is a misleading guide to the model’s expressive power, because the embedding layer is essentially a lookup table rather than a computational bottleneck.

What is *not* in the table is what makes DeBERTa distinctive. Disentangled attention, the architectural feature that sets DeBERTa apart from the other two, does not appear in this table because it is not a shape parameter. It changes the formula by which attention is computed within each block (Figure 4), and its effect is visible in quality benchmarks rather than in parameter counts. This is also why DeBERTa-v3-xs — the smallest model by backbone parameters in our comparison — is competitive with RoBERTa-base, which is roughly four times larger by the same measure.

Property	DeBERTa-v3 (xs)	RoBERTa-base	ELECTRA-large
Transformer layers N	12	12	24
Hidden size H	384	768	1024
Attention heads A	6	12	16
Per-head dimension H/A	64	64	64
FFN intermediate size	1 536	3 072	4 096
Vocabulary size	128 000	50 265	30 522
Max. position embeddings	512	512	512
Tokeniser	SentencePiece (BPE)	Byte-level BPE	WordPiece
Backbone parameters	~22M	~86M	~335M
Embedding parameters	~48M	~39M	~31M
Total parameters	~70M	~125M	~335M
Pretraining objective	RTD	Dynamic MLM	RTD (discriminator)

Table 1: Side-by-side architectural specification of the three encoders compared in this study. RTD stands for replaced-token detection, the ELECTRA-style pretraining objective that DeBERTa-v3 also adopts.

5 Results

5.1 Validation performance

Table 2 summarises the best validation log loss and accuracy achieved by each backbone.

Backbone	Val. log loss	Val. accuracy	Best epoch
DeBERTa-v3 (xs)	1.0597	0.454	8
RoBERTa-base	1.0492	0.464	2
ELECTRA-large	1.0974	0.349	—
Uniform baseline ($\log 3 \approx 1.0986$)	1.0986	0.333	—

Table 2: Best validation performance per backbone. The uniform-prediction baseline ($\log 3$) is included as a sanity floor.

RoBERTa-base achieved the lowest validation log loss, slightly ahead of DeBERTa-v3-xs despite being roughly five times larger. ELECTRA-large did not converge to a useful state under the chosen optimisation regime; its validation loss remained essentially at the uniform baseline. We return to this in the discussion.

5.2 Predicted probabilities on the test set

Because the competition is scored on log loss rather than accuracy, the actual probability distributions the models output are at least as informative as their single-class predictions. The test set in this run contained three examples, all of which were scored by each model. Table 3 shows the per-class probabilities each model assigned.

Test id	Model	$P(\text{A wins})$	$P(\text{B wins})$	$P(\text{tie})$
136060	DeBERTa-v3 (xs)	0.267	0.234	0.499
	RoBERTa-base	0.033	0.814	0.153
	ELECTRA-large	0.349	0.335	0.316
211333	DeBERTa-v3 (xs)	0.363	0.335	0.302
	RoBERTa-base	0.298	0.352	0.349
	ELECTRA-large	0.349	0.335	0.316
1233961	DeBERTa-v3 (xs)	0.412	0.299	0.289
	RoBERTa-base	0.227	0.470	0.303
	ELECTRA-large	0.349	0.335	0.316

Table 3: Per-class probabilities assigned by each model to the three test examples. The **bold** entry in each row is the argmax class. Each row sums to 1 up to rounding (the final layer is softmax over the three classes).

This table is worth reading in some detail. Recall that the competition submission format is exactly this: for each test id, the three probabilities must be supplied, and they will be scored against a one-hot ground truth under log loss.

What each row means. Take id 136060 as an illustration. The input was a fixed (prompt, response A, response B) triple from the test set, which is identical for all three models. The DeBERTa model returned a near-tie distribution leaning towards tie: it judges the two responses comparable, with a slight edge to model A. RoBERTa, in contrast, returned a sharply confident prediction that model B wins ($P = 0.814$). The two models clearly read the example differently.

Sharpness vs. caution. Across all three examples, RoBERTa’s distributions are noticeably sharper than DeBERTa’s: it concentrates more mass on a single class, sometimes much more. Sharp predictions are heavily rewarded by log loss when they are right and heavily penalised when they are wrong. The validation results (Table 2) suggest RoBERTa is right often enough to come out ahead, but the variance is higher: a confidently wrong prediction at $P = 0.81$ contributes $-\log(1 - 0.81) \approx 1.66$ to the loss on that example, where a more measured DeBERTa prediction at $P = 0.27$ for a wrong class contributes only $-\log(1 - 0.27) \approx 0.31$.

ELECTRA outputs the same distribution everywhere. The most striking observation in Table 3 is that ELECTRA returns *identical* probabilities for all three test examples: (0.349, 0.335, 0.316). This is the visible symptom of the failure diagnosed in Table 2: the model never moved away from a trivial constant prediction during training, and so it produces the same output regardless of input. The probabilities themselves are not random: they reflect the empirical class frequencies in the training data (model A wins is the most common outcome, model B wins second, tie least common), which is exactly what a classifier reduces to when it fails to extract signal from its inputs.

5.3 Discussion of the comparison

The combination of Table 2 and Table 3 gives a clearer picture than either alone. A few points stand out.

First, model size is not a reliable predictor of performance on this task once a basic capacity threshold is met. The smallest model (DeBERTa-v3-xs) is competitive with the second-largest (RoBERTa-base), and the largest (ELECTRA-large) is the worst of the three under the chosen training regime.

Second, all three working models settle into validation log losses in roughly the same band (1.04–1.06), only modestly better than the uniform baseline (1.099). This is consistent with the broader observation that human preferences over LLM responses are inherently noisy, and that a Siamese-style architecture — which never lets the encoder attend *across* the two responses — leaves room for improvement that no choice of backbone within this family can recover.

Third, ELECTRA-large’s failure is an optimisation issue, not a representational one. The model has more than enough capacity for the task; it simply did not receive a learning-rate schedule that allowed the classification head to draw the backbone away from its pretrained initialisation. Adding linear warmup, increasing the peak learning rate to the range typically used for large models, or briefly training the head alone before unfreezing the backbone would all be expected to fix this.

6 Future Work

The most promising next step is to depart from the Siamese formulation in favour of a single-sequence cross-encoder, in which the prompt and both responses are concatenated into one sequence and processed jointly. This would let self-attention compare the two responses directly during encoding, which is a much stronger inductive bias for a preference task. Other worthwhile directions include increasing the sequence length to capture more of each response, switching to

a multilingual encoder so the non-ASCII filtering step can be removed, and initialising from a pretrained reward model rather than from a vanilla language encoder.

References

- [1] W.-L. Chiang et al., *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference*, arXiv:2403.04132, 2024.
- [2] A. Vaswani et al., *Attention Is All You Need*, NeurIPS, 2017.
- [3] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, R. Shah, *Signature Verification using a “Siamese” Time Delay Neural Network*, NeurIPS, 1993.
- [4] P. He, J. Gao, W. Chen, *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing*, arXiv:2111.09543, 2021.
- [5] Y. Liu et al., *RoBERTa: A Robustly Optimized BERT Pretraining Approach*, arXiv:1907.11692, 2019.
- [6] K. Clark, M.-T. Luong, Q. V. Le, C. D. Manning, *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators*, ICLR, 2020.
- [7] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, A. Bordes, *Supervised Learning of Universal Sentence Representations from Natural Language Inference Data*, EMNLP, 2017.